



our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Colb Finance Multicall

SECURITY REVIEW

Date: 27 February 2026

CONTENTS

1. About Shieldify	3
2. Disclaimer	3
3. About Colb Finance - Multicall	3
4. Risk classification	4
4.1 Impact	4
4.2 Likelihood	4
5. Security Review Summary	5
5.1 Protocol Summary	5
5.2 Scope	5
6. Findings Summary	5
7. Findings	6

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Colb Finance - Multicall

Colb is the first native non-custodial tokenization solution that enables peerless access to Swiss-grade wealth management strategies, pre-IPO opportunities, and premium investment funds. It offers a bankruptcy-remote Trust structure and native ownership of real-world assets, all on-chain.

Colb reduces the entrance threshold to such investments by removing constraints such as the minimum investment amount to get exposure to them. The protocol is designed with security at its core, boasting compliance with Swiss regulations and DeFi composability. Colb envisions a future rooted in transparency where every individual has equitable access to premium RWA investments.

Overview

Multicall combines multiple financial operations into a single transaction. Operations either all succeed or all fail together, preventing partial execution states.

Executing operations separately creates risk. A user might successfully mint \$USC tokens, then fail to deposit them into a strategy. The user ends up holding tokens they didn't want, requiring additional transactions to resolve.

Multicall eliminates this by ensuring operations execute atomically. If any operation fails, the entire transaction reverts, leaving the user's state unchanged if the calls require atomicity.

Workflow

Multicall coordinates operations through specialized adapters. Each adapter handles a specific function - minting tokens, depositing into strategies, or other operations which are supported by the registered adapters in the **Multicall**.

When a user submits a batch request, **Multicall** validates permissions, routes each operation to the appropriate adapter, and ensures all operations complete successfully. If any operation fails, the entire batch reverts, or if the caller wants, the operations could continue even after a failed operation.

The system preserves the original user's identity throughout all adapter calls, ensuring tokens and shares are issued to the correct recipient.

Adapters

- **EngineAdapter** handles \$USC minting. It accepts collateral, calculates fees, and mints \$USC tokens to a specified receiver (which can be another adapter or another whitelisted account). The EngineAdapter only expects the receiver after a mint to be another adapter whitelisted in the adapter and in the **Multicall** contract. This guarantees that the daily limits set for the adapter or the fee rates won't be misused. In the case of mint + deposit in a strategy, the user will pay an extra cancellation fee for the cancellation of the deposit. The daily limits are not usually applied and in case they are, the adapter would be disabled.
- **StrategyAdapter** handles deposits to Liquid Strategies. It accepts tokens from previous operations, deposits them into the specified strategy, and issues shares to the original user. The adapter expects to hold the funds for deposit before the operation.

Usage of Multicall

- Users must be whitelisted with appropriate access levels and approve collateral tokens before batching.
- For an adapter to be onboarded, it should be whitelisted by the administrators.
- The adapter should verify that it is called by **Multicall** in its internal logic and use **Multicall** to retrieve the original user's identity.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** - results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** - results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** - losses will be limited but bearable - and covers vectors similar to grieving attacks that can be easily repaired

4.2 Likelihood

- **High** - almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** - still relatively likely, although only conditionally possible
- **Low** - requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 5 days, with a total of 80 hours dedicated to the audit by two researchers from the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium and five Low severity issues. They're mainly related to Multicall token sweeping, share-asset conversion inconsistencies, withdrawal handling, and multicall configuration/ETH handling risks.

The Colb Finance team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Colb Finance - Multicall
Repository	contracts-v2
Type of Project	RWAs, Pre-IPO, Liquids
Security Review Timeline	5 days
Review Commit Hash	fdc41c8373851c0ad6bb384f2f2a35dd4c16b5da
Fixes Review Commit Hash	f7dfcc4c7d85c15374eb3c238029cc0e23c09279

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
contracts/multicall/Multicall.sol	42
contracts/multicall/adapters/BaseAdapter.sol	19
contracts/multicall/adapters/EngineAdapter.sol	37
contracts/multicall/adapters/StrategyAdapter.sol	28
contracts/liquid/abstract/AsyncStrategy.sol	164
contracts/liquid/abstract/CoreStrategy.sol	359
contracts/liquid/RWAlliquidStratey.sol	34
Total	683

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **1**
- **Low** issues: **5**

ID	Title	Severity	Status
[M-01]	<code>StrategyAdapter</code> Can Be Used to Sweep Any Token Balance Held by the Adapter Into Strategy Shares for the Multicall Caller	Medium	Fixed
[L-01]	Assymetry Between Shares-to-Asset Conversion and Vice-Versa	Low	Acknowledged
[L-02]	The <code>processWithdrawals()</code> Does Not Check Withdrawal Pause Before Processing	Low	Acknowledged
[L-03]	The <code>setLiquidAssets()</code> in <code>CoreStrategy</code> Can Reset the Remaining Withdrawal Budget Mid-Epoch	Low	Acknowledged
[L-04]	Configuration State Variables for <code>Multicall</code> Are Immutable	Low	Acknowledged
[L-05]	Excess ETH Sent to <code>Multicall</code> Will Become Stuck Forever	Low	Fixed

7. Findings

[M-01] `StrategyAdapter` Can Be Used to Sweep Any Token Balance Held by the Adapter Into Strategy Shares for the Multicall Caller

Severity

Medium Risk

Description

The `StrategyAdapter.deposit()` assumes the adapter already holds the tokens to deposit and does not verify that those tokens belong to the caller or were received within the same Multicall execution.

The function simply approves the specified token from the adapter's own balance to the strategy and calls `deposit(amount, receiver)` where `receiver` is the `Multicall` caller.

This means that any Multicall-eligible user can call `deposit()` specifying any amount up to the adapter's full token balance and receive the resulting strategy shares themselves.

Location of Affected Code

File: [contracts/multicall/adapters/StrategyAdapter.sol#L45-L55](#)

```
function deposit(address strategy, uint256 amount, address _token)
    external onlyMulticall {
        require(_token != address(0), ZeroAddress());
        require(amount != 0, ZeroAmount());
        require(strategies[strategy], StrategyNotWhitelisted());
        // approve with the exact amount
        IERC20(_token).forceApprove(strategy, amount);
        // deposit in `strategy`
        ICoreStrategy(strategy).deposit(amount, _sender());
        // nullify approval
        IERC20(_token).forceApprove(strategy, 0);
    }
}
```

Impact

Any ERC20 tokens held by `StrategyAdapter` can be converted into strategy shares owned by a malicious user. Direct and permanent loss of those funds for the original sender.

Recommendation

Add a `safeTransferFrom()` at the start of `StrategyAdapter.sol##deposit()` to pull tokens from the original user rather than relying on the adapter's existing balance. Or track and restrict deposits to tokens received within the same Multicall execution.

Team Response

Fixed.

[L-01] Assymetry between shares-to-asset conversion and vice-versa

Severity

Low Risk

Description

The `_convertToShares()` and `_convertToAssets()` are not exact inverses of each other. The asymmetry comes from `_convertToShares()` computing an intermediate "unit quote" using integer division (truncation), then dividing by that truncated value.

- USC decimals: **6**
- Oracle price decimals: **8**
- Strategy share decimals: set to USC decimals (`_decimals = USC.decimals()` in `CoreStrategy._CoreInit`)

** Conversion math (integer rounding)**

Shares Assets (`_convertToAssets()`)

Single `mulDiv` (one truncation):

```
assets = floor(price * shares / 108)
```

Assets Shares (`_convertToShares()`)

Two-step calculation (two truncations):

```
Q      = floor(price * 106 / 108) = floor(price / 100)
shares = floor(assets * 106 / Q)
```

When $\text{price} \% 100 \neq 0$, `Q` is strictly smaller than the exact unit quote $(\text{price} / 100)$. Since `Q` is used as a divisor, a smaller `Q` results in **slightly more shares minted** than a single-step inverse calculation would produce.

Location of Affected Code

File: [CoreStrategy.sol#L724](#)

```
function _convertToShares(uint256 _assets) internal view returns (uint256
    shares) {
    uint256 _quote = oracle.getQuoteRaw(10 ** _decimals);
    if (_quote == 0) return 0;
    return _assets.mulDiv(10**_decimals, _quote, MathUpgradeable.Rounding
        .Down);
}
```

File: [CoreStrategy.sol#L733](#)

```
function _convertToAssets(uint256 _shares) internal view returns (uint256
    assets) {
    return oracle.getQuoteRaw(_shares);
}
```

File: [LSPPriceOracle.sol#L72](#)

```
function getQuoteRaw(uint256 amount) public view returns (uint256) {
    // code
    return Math.mulDiv(uint256(_answer), amount, 10 ** aggregator.
        decimals());
}
```

Impact

- **Correctness:** conversions are not symmetric; some deposits can receive slightly more shares than the exact inverse of `convertToAssets()` would imply.
- **Scale:** the rounding edge is small per unit, but it scales with deposit size. Because the intermediate quote is truncated at the 6-decimal boundary, the relative error is on the order of ~1 ppm ($1e-6$) for prices around \$1.

Proof of Concept

Assume:

- USC decimals = 6 (so `2_000_000` means **2.000000 USC**)
- Oracle decimals = 8
- `price (P) = 100_000_099` [1.000000099 with 8 decimals]

Step 1 — compute the truncated unit quote `Q`:

```
Q = getQuoteRaw(10^6)
  = floor(P × 10^6 / 10^8)
  = floor(1_000_000.99)
  = 1_000_000
```

Step 2 — convert `assets = 2_000_000` (**2.000000 USC**) to shares:

```
shares = floor(2_000_000 × 10^6 / 1_000_000)
        = 2_000_000
```

Exact (non-truncated) unit quote would be **1_000_000.99**, so:

```
shares_exact = floor(2_000_000 × 10^6 / 1_000_000.99)
              = 1_999_998
```

Over-issuance: `2_000_000 - 1_999_998 = 2` share-units [2 units at 6 decimals = 0.000002 shares].

Step 3 — convert all `shares = 2_000_000` back to assets:

```
assets_back = floor(P × 2_000_000 / 10^8)
             = floor(2_000_001.98)
             = 2_000_001
```

Result (unit-correct):

- Deposit: 2.000000 USC
- Redeem (conversion-only): 2.000001 USC
- Gain: 0.000001 USC [1 micro-USC]

Recommendation

Avoid creating a truncated per-unit quote inside `_convertToShares()`. Instead, compute the inverse quote in a single `mulDiv` using the same `(price, decimals)` inputs that the oracle uses.

Because `CoreStrategy` only depends on `ILSPPriceOracle`, a practical fix is to extend the oracle with an inverse-quote method and use it from `_convertToShares()`:

```
// In ILSPPriceOracle / LSPriceOracle
function getInverseQuoteRaw(uint256 quote) external view returns (uint256
    amount);

// In LSPriceOracle implementation
function getInverseQuoteRaw(uint256 quote) public view returns (uint256)
{
    if (quote == 0) revert InvalidAmount();
    (, int256 answer,,, ) = aggregator.latestRoundData();
    if (answer <= 0) revert InvalidPrice();
    return Math.mulDiv(quote, 10 ** aggregator.decimals(), uint256(answer
        ));
}
```

Then implement `_convertToShares(_assets)` as:

```
return oracle.getInverseQuoteRaw(_assets);
```

This makes both directions structurally consistent (one `mulDiv`, one truncation).

Team Response

Acknowledged.

[L-02] The `processWithdrawals()` Does Not Check Withdrawal Pause Before Processing

Severity

Low Risk

Description

The Controller enforces withdrawal pause windows (global and per-strategy) via `checkWithdrawal()`, which reverts when withdrawals are paused. This check is invoked in `_validateWithdrawalRequest()`, which is called by `redeem()` and `redeemInstant()` when users create withdrawal requests.

However, `processWithdrawals()` — the operator function that executes pending withdrawal requests — does not call `checkWithdrawal()`. As a result, the pause is enforced only at request creation time, not at processing time.

Location of Affected Code

File: [contracts/liquid/abstract/CoreStrategy.sol#L378-L429](#)

```
function processWithdrawals(uint256[] calldata ids) external virtual {
    RoleChecker.hasRole(kernel.roleManager, operatorRole());
    oracle.checkIfPriceIsStale();
    // @audit No checkWithdrawal() call - pause not enforced at
    // processing time
    uint256 timeframe = getTimeframe(block.timestamp);
    IController _controller = IController(kernel.controller);
    // code
}
```

File: [contracts/liquid/abstract/CoreStrategy.sol#L725-L743](#)

```
function _validateWithdrawalRequest(address _sender, address _receiver,
    uint256 _shares) internal view {
    IController _controller = IController(k.controller);
    _controller.checkWithdrawal(address(this)); // Present for redeem/
    redeemInstant
    // code
}
```

Impact

When withdrawals are paused (e.g., during an emergency or maintenance), users cannot create new withdrawal requests because `redeem()` and `redeemInstant()` revert. However, the operator can still process existing pending requests via `processWithdrawals()`, allowing \$USC to be minted and sent to receivers. The pause is therefore incomplete at the contract level; the protocol relies on the operator not calling `processWithdrawals()` during a pause rather than enforcing it on-chain.

Recommendation

Add `checkWithdrawal()` at the start of `processWithdrawals()` so the pause is enforced at both request creation and processing:

```
function processWithdrawals(uint256[] calldata ids) external virtual {
    RoleChecker.hasRole(kernel.roleManager, operatorRole());
    oracle.checkIfPriceIsStale();
    IController(kernel.controller).checkWithdrawal(address(this));
    uint256 timeframe = getTimeframe(block.timestamp);
    // code
}
```

Team Response

Acknowledged.

[L-03] The `setLiquidAssets()` in `CoreStrategy` Can Reset the Remaining Withdrawal Budget Mid-Epoch

Severity

Low Risk

Description

The `processWithdrawals()` treats `liquidAssets[timeframe]` as an available budget and decrements it as withdrawals are processed. However, `setLiquidAssets()` overwrites `liquidAssets[timeframe]` with a fresh value for the same epoch (`timeframe = getTimeframe(block.timestamp)`), ignoring any amounts already consumed earlier in the epoch.

This breaks the invariant “liquidAssets is the remaining withdrawable amount for the epoch” and allows the epoch budget to be effectively re-expanded mid-epoch by an `assetManagerRole()` holder.

Location of Affected Code

File: [contracts/liquid/abstract/CoreStrategy.sol#L226-L241](#)

```
function setLiquidAssets(uint256 _liquidAssets) external {
    RoleChecker.hasRole(kernel.roleManager, assetManagerRole());

    oracle.checkIfPriceIsStale();

    uint256 _totalAssets = totalAssets();

    if (_liquidAssets > _totalAssets) {
        revert LAExceedTotalAssets(_liquidAssets, _totalAssets);
    }

    uint256 timeframe = getTimeframe(block.timestamp);
    liquidAssets[timeframe] = _liquidAssets;

    emit LiquidAssetsUpdated(_liquidAssets, timeframe);
}
```

File: [contracts/liquid/abstract/CoreStrategy.sol#L378-L429](#)

```

function processWithdrawals(uint256[] calldata ids) external virtual {
    RoleChecker.hasRole(kernel.roleManager, operatorRole());

    oracle.checkIfPriceIsStale();

    uint256 timeframe = getTimeframe(block.timestamp);

    IController _controller = IController(kernel.controller);

    for (uint256 i = 0; i < ids.length; i++) {
        uint256 id = ids[i];

        Withdrawal memory withdrawal = withdrawals[id];

        if (withdrawal.sender == address(0)) {
            revert InvalidRequest();
        }
        if (withdrawal.status != ICoreStrategy.Status.Pending) {
            revert InvalidState();
        }
        if (_controller.isBlacklisted(withdrawal.sender) || _controller.isBlacklisted(withdrawal.receiver)) {
            revert AddressBlacklisted();
        }

        uint256 shares = withdrawal.shares;

        uint256 assets = _convertToAssets(shares);

        if (assets == 0) {
            revert InvalidRequest();
        }

        if (assets > liquidAssets[timeframe]) {
            revert InsufficientLiquidAssets();
        }
    }
}

```

```

uint256 fee = getFee(ActionType.REDEEM, assets, withdrawal.sender
);

withdrawals[id].status = ICoreStrategy.Status.Processed;

pendingWithdrawalShares[withdrawal.sender] -= shares;

// deduct `assets` before fees
liquidAssets[timeframe] -= assets;

_processWithdrawal(withdrawal.receiver, address(this), assets,
    shares, fee);

emit ProcessWithdrawal(id, withdrawal.sender, withdrawal.receiver
    , assets, shares, fee, treasury, feeTreasury);
    }
}

```

Impact

The per-epoch withdrawal limit can be doubled by resetting `liquidAssets[timeframe]` after withdrawals have already been processed in the same epoch.

Recommendation

Change `setLiquidAssets()` to update the remaining budget safely (e.g., set a separate `epochTotalLiquidAssets[timeframe]` and compute remaining as `total - spent` or only allow increasing/decreasing relative to already-consumed amounts) or enforce something like “only callable once per epoch.”

Team Response

Acknowledged.

[L-04] Configuration State Variables for `Multicall` Are Immutable

Severity

Low Risk

Description

`Multicall` sets three core configuration parameters – `whitelist`, `roleManager` and `accessLevel`. They are set only in the constructor and provide no setter functions to update them afterwards.

If any of these values need to change (e.g., whitelist replacement, role manager migration or access level adjustment), the only option is redeploying a new Multicall contract and reconfiguring all adapters and integrations.

Additionally, the `IMulticall` interface explicitly declares

`AccessLevelUpdated(uint256 oldLevel, uint256 newLevel)`. This event can never be emitted because no setter exists. This is a clear interface/implementation mismatch.

Location of Affected Code

File: [contracts/multicall/Multicall.sol#L37-L42](#)

```
constructor(address _whitelist, address _roleManager, uint256
    _accessLevel) {
    require(_whitelist != address(0) && _roleManager != address(0),
        ZeroAddress());
    whitelist = IWhitelist(_whitelist);
    roleManager = _roleManager;
    accessLevel = _accessLevel;
}
```

Recommendation

Add admin setters for `whitelist`, `roleManager` and `accessLevel` [with appropriate events].

Team Response

Acknowledged.

[L-05] Excess ETH Sent to `Multicall` Will Become Stuck Forever

Severity

Low Risk

Description

The `Multicall.multicall()` is payable and forwards `calls[i].value` to each adapter call, but it does not verify that the total forwarded ETH equals `msg.value` and provides no withdrawal mechanism.

If a user sends more ETH than the sum of forwarded values, the remainder is locked forever in the `Multicall` contract.

Location of Affected Code

File: [contracts/multicall/Multicall.sol#L63-L82](#)

```

function multicall(Call[] calldata calls) external payable nonReentrant {
    require(calls.length > 0, EmptyMulticall());
    require(whitelist.whitelist(msg.sender) >= accessLevel,
        AccountNotWhitelisted());

    sender = msg.sender;

    for (uint256 i; i < calls.length; ++i) {
        address to = calls[i].to;
        require(adapters[to], AdapterNotWhitelisted());

        (bool success, bytes memory returnData) = to.call{value: calls[i].value}(calls[i].data);
        if (!success) {
            assembly {
                revert(add(returnData, 32), mload(returnData))
            }
        }
    }

    sender = address(0);
}

```

Impact

ETH overpayment to `Multical` results in permanent fund loss, as the contract has no ETH recovery path.

Proof of Concept

1. User calls `multical` with `msg.value = 1 ETH`
2. All `calls[i].value = 0`
3. No ETH is forwarded to adapters
4. `1 ETH` remains in `Multical` with no withdrawal function

Recommendation

Two possible mitigation: require `msg.value == sum(calls[i].value)` or add an admin or user ETH recovery mechanism.

Team Response

Fixed.

our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Thank you!

